



AI 模型与工程

从会用模型到能交付系统

HackProve World 2026 白帽世界大会中国·澳门

主题 08 02:35:12-03:03:09

资料范围：本主题 transcript、rel/abs SRT、pdf outline contract、official_references.md。

目录

1	主题定位：会用模型只是入口	3
2	从提示词到 workflow	3
2.1	任务拆解：把目标变成步骤	3
2.2	工具调用：沿用主题 02 的定义	4
2.3	状态管理：安全任务怕丢细节	4
3	模型、数据与工具的系统关系	5
3.1	模型 M：能力底座	5
3.2	数据 D：领域经验的容器	5
3.3	工具：行动能力的边界	5
4	模型工程与交付工程的分工	6
4.1	模型选择：质量、成本、延迟、可靠性	6
4.2	模型工程：让模型更懂任务	7
4.3	交付工程：让系统能被使用和问责	7
5	安全工程要求：沿用主题 03 的控制概念	7
5.1	风险矩阵 A/T/C/E	7
5.2	权限：底层控制优先于口头约束	8
5.3	日志、回滚与人工接管	8
6	评估：从演示效果到生产可信	8
6.1	评估对象	8
6.2	指标组合	9
7	开放工作台与生态	9
7.1	开放的两层含义	9
7.2	工作台的交付形态	10
8	交付清单	10
8.1	输入清单	10
8.2	处理清单	10
8.3	输出清单	11
9	官方参考	11
10	配图接口	11

11 结语**11**

摘要

本讲义讨论主题 08 的核心问题：安全团队已经会使用模型，下一步要把模型放进可交付的工程系统。现场演讲以一款面向白帽和渗透测试的 AI workbench 为例，反复强调模型能力、工具调用、上下文管理、权限控制、人工验证和端到端交付之间的组合关系。

本讲义沿用主题 02 对 Agent/代理与工具调用的定义：Agent 是能围绕目标进行多步推理、调用工具、维护状态并产出行动的系统组件；工具调用是把自然语言意图转成结构化函数、API、命令或插件动作的过程。讲义同时复用主题 03 的审计控制概念，用资产 A 、威胁 T 、控制 C 、证据 E 来组织权限、日志、回滚和人工接管。主题 08 的重点落在模型 M 、数据 D 、工具、评估与交付流程如何合成一个工程系统。

1 主题定位：会用模型只是入口

演讲者开场询问听众日常使用哪些 AI 应用，并提到 Claude Code、Codex、GPT 等偏编程方向的工具。随后现场把问题收束到安全领域：AI 已经能写代码、做分析、执行任务，真正困难的地方在于如何把它融入渗透测试和漏洞挖掘的完整流程，让它理解上下文、调用工具、保留证据，并让白帽能够判断结果是否正确。

这段内容的工程含义很直接：模型本身只是系统中的一个部件。单独让模型回答问题，像把一台高性能发动机放在桌上；交付系统需要底盘、刹车、仪表、油路、驾驶规则和维修手册。对于 AI 安全工程，这些配套分别对应数据、工具、权限、日志、评估、人工复核和发布流程。

本章主张

“会用模型”解决的是单次问答或单次执行；“能交付系统”要求模型 M 、数据 D 、工具、评估、审计控制和人工接管形成闭环。工程能力决定系统能否持续、稳定、可解释地完成安全任务。

演讲中有一句可作为本章锚点的判断：模型决定执行效果的下限，工程决定执行效果的上限。该判断很硬，也很现实。低能力模型很难撑起复杂任务；高能力模型若缺少上下文管理、工具边界和证据链，也会把错误快速放大。

2 从提示词到工作流

2.1 任务拆解：把目标变成步骤

白帽任务通常从模糊目标开始，例如“检查这个站点是否存在可利用漏洞”。人类测试者会自然地拆成信息收集、入口枚举、请求观察、业务逻辑推断、候选漏洞验证、证据留存和报告撰写。模型要进入这个流程，也必须把目标拆成可执行步骤。

可以把任务表示为一组有依赖关系的动作：

$$G \rightarrow \{s_1, s_2, \dots, s_n\}, \quad s_i = (input_i, tool_i, output_i, evidence_i)$$

其中 G 是目标, s_i 是第 i 个步骤, 包含输入、工具、输出和证据。工程系统要管理的不只是 s_i 是否完成, 还要管理步骤之间的依赖、失败重试、人工审批和证据归档。若某一步缺少证据, 后续结论就会变成悬空判断。

2.2 工具调用：沿用主题 02 的定义

主题 02 已定义工具调用：模型或代理把意图转成结构化函数、API、命令或插件动作的过程。本章沿用该定义。演讲中的白帽 workbench 设想, 把浏览器、抓包工具、扫描器、MCP、Skill 和 Agent 接到同一个工作台, 让 AI 在明确边界内串起流程。

工具调用链可写为：

$$\langle request, context, tool_i, args_i, result_i, evidence_i, approval_i \rangle_{i=1}^n$$

这条链里有三个工程要点：

1. **参数可见**：工具输入要结构化, 避免模型把模糊意图直接交给高风险工具。
2. **结果可证**：每次调用要产生可回放证据, 例如请求响应、截图、日志或 diff。
3. **审批可插入**：读操作、写操作、破坏性操作和敏感数据访问要进入不同权限层。

2.3 状态管理：安全任务怕丢细节

现场访谈中, Trevor 强调渗透测试的关键细节很小, 可能是某个隐藏参数、一个 token、一个接口异常或一段前端 JavaScript 逻辑。普通摘要只抓大意, 在安全测试里会漏掉路径。漏掉一个细节, 可能就丢掉整条漏洞链。

因此, 工程系统要把上下文分层保存：

上下文层	内容	工程要求
任务目标	范围、资产、禁止动作、验收标准	固定在任务单中, 不能随对话漂移
工作记忆	当前步骤、候选假设、临时观察	可压缩, 但必须保留安全关键字段
证据记忆	请求响应、截图、日志、代码片段	不可随意改写, 需关联来源和时间
经验记忆	Skill、历史案例、误报规则、复现模板	版本化管理, 需经过评估再沉淀

上下文压缩像整理案卷。普通办公场景可以压缩成“事情大概如此”；渗透测试要保留案卷中的针脚、指纹和时间戳。压缩策略若只追求短, 会让 Agent 变得轻巧, 也会让它失去追踪漏洞链的能力。

3 模型、数据与工具的系统关系

3.1 模型 M：能力底座

在本讲义中， M 表示模型，包含基础模型、领域增强模型、微调模型和评估模型。模型能力决定系统能否理解代码、业务逻辑、协议和安全语境。演讲者提到，白盒扫描若只靠规则匹配、污点溯源或传统扫描器，误报会很高；引入 AI 后，系统有机会理解业务代码中隐藏的漏洞。

模型能力可以从四个维度评估：

- **理解能力**：能否读懂代码、请求、响应、日志和业务规则。
- **规划能力**：能否把任务拆成合理步骤，并在失败后调整路径。
- **工具协作能力**：能否正确选择工具、组织参数、解释返回结果。
- **证据意识**：能否把结论绑定到可复核证据，而非只输出判断。

3.2 数据 D：领域经验的容器

D 表示数据集，包含训练、微调、评估、检索和日志数据。主题 08 中的数据重点放在工程数据如何服务安全任务：真实漏洞案例、代码审计样本、历史误报、复现步骤、报告模板、工具调用日志，都会影响系统输出。

演讲者提到通过真实漏洞数据做 RAG，并对模型做过调控或微调，目标是让系统更精准地从代码中挑出真实问题。这说明 D 的价值在于把白帽经验转成可检索、可评估、可复用的材料。数据像厨房里的高汤，模型像厨师；高汤若来自真实食材，菜会有底味。高汤若混入错误样本，输出也会被带偏。

$$Capability = f(M, D, Tools, Workflow, Controls)$$

这里的 $Capability$ 是系统能力。单独提升 M 有帮助，清洗 D 、约束工具、优化工作流和补齐控制同样关键。

3.3 工具：行动能力的边界

工具决定系统能接触现实世界的哪些部分。演讲中提到浏览器、抓包、扫描器、Burp 类工具、NMAP、SQLMAP、MCP 和 Skill。工程系统要把这些工具视为不同风险等级的行动接口。

工具类型	典型用途	控制要求
浏览器/无头浏览器	页面观察、交互复现、截图留证	记录访问路径，限制敏感操作

抓包与代理工具	请求捕获、参数分析、复现证据	保留请求响应，屏蔽敏感字段
扫描器	资产枚举、漏洞候选发现	限制范围、速率和破坏性测试
代码分析工具	白盒扫描、调用链分析、污点追踪	绑定仓库版本，记录命中规则和证据
MCP/Skill/Agent	外部能力接入、经验复用、	白名单、接口文档、权限分层和审计流程编排

工具越强，控制越要硬。一个只会读文件的工具风险有限；一个能写文件、改数据库、外发网络请求或执行命令的工具，必须经过权限、审批和日志控制。

4 模型工程与交付工程的分工

4.1 模型选择：质量、成本、延迟、可靠性

模型选择不是排行榜采购。安全工程要按任务选择模型，并把质量、成本、延迟和可靠性放在同一张表里看。

维度	问题	交付影响
质量	是否能正确理解安全语境和证据	决定误报、漏报与报告可信度
成本	单次调用、长上下文、工具调用是否可承受	决定能否规模化运行
延迟	多步任务是否会拖慢测试节奏	决定交互体验和自动化吞吐
可靠性	同类任务输出是否稳定	决定能否进入标准流程
可控性	是否支持结构化输出、工具约束和追踪	决定审计与回滚难度

可以用一个简化评分函数描述模型选择：

$$Score(M) = w_qQ - w_cCost - w_lLatency + w_rReliability + w_sSafety$$

其中 Q 是质量， $Cost$ 是成本， $Latency$ 是延迟， $Reliability$ 是可靠性， $Safety$ 是安全可控性， w 为业务权重。高风险安全任务通常会提高 w_r 与 w_s ，因为稳定和可控比短期速度更重要。

4.2 模型工程：让模型更懂任务

模型工程关注模型如何理解任务。常见手段包括提示模板、检索增强、微调、评估样例和领域词表。主题 08 中，安全领域的模型工程重点包括：

- 用真实漏洞案例构建检索库，让模型看到接近业务的材料；
- 用历史误报构建反例，让模型学会收敛结论；
- 用报告模板约束输出，避免证据、影响和修复建议散乱；
- 用评估集测试代码理解、漏洞复现和证据引用能力；
- 用上下文压缩策略保留参数、token、接口异常等关键细节。

4.3 交付工程：让系统能被使用和问责

交付工程关注系统如何进入真实工作。它包含权限、日志、版本、发布、回滚、人工接管、客户交付和责任边界。模型工程解决“模型懂不懂”，交付工程解决“系统能不能被信任地用”。

下限与上限

模型能力给出执行效果的下限：模型太弱，复杂任务难以成立。工程能力给出执行效果的上限：缺少 workflow、评估、权限和审计，再强的模型也会困在演示阶段。

5 安全工程要求：沿用主题 03 的控制概念

5.1 风险矩阵 A/T/C/E

主题 03 已定义风险矩阵。本章沿用其控制语言：*A* 表示资产，*T* 表示威胁，*C* 表示控制，*E* 表示证据。模型工程进入交付阶段后，每个功能都要能落到这四格。

维度	主题 08 中的含义
<i>A</i> 资产	模型、代码仓库、抓包数据、客户系统、工具凭证、报告草稿、上下文记忆。
<i>T</i> 威胁	错误工具调用、越权执行、上下文污染、敏感信息泄露、误报写入交付物、破坏性动作。
<i>C</i> 控制	最小权限、工具白名单、审批门禁、沙箱、日志、评估、回滚、人工接管。

E 证据 请求响应、截图、工具日志、模型输出、审批记录、测试结果、报告版本。

5.2 权限：底层控制优先于口头约束

演讲者明确提到，只靠提示词限制 AI 权限并不可靠。上下文变长或被污染后，模型可能遗忘先前约束。工程系统应从底层规定哪些工具能调用、哪些工具不能调用、哪些工具调用前必须询问用户。

权限分层可以这样设计：

1. **观察层**：查看文件、页面、日志、请求响应，默认只读。
2. **分析层**：生成假设、整理报告、解释工具输出，不直接改动环境。
3. **执行层**：调用扫描器、浏览器、抓包工具，限制范围和速率。
4. **变更层**：写文件、修改配置、编辑代码、提交请求，必须显式审批。
5. **高危层**：删除、外发敏感数据、修改数据库、生产环境利用，默认禁止或隔离执行。

这种分层把“模型是否听话”改成“系统是否给了它可越界的手”。安全工程里，手比嘴更重要；能不能做某件事，要由权限系统决定。

5.3 日志、回滚与人工接管

一个能交付的 AI workbench 必须记录三类日志：

- **意图日志**：用户目标、任务范围、禁止动作、审批意见。
- **行动日志**：工具名称、参数、输出、错误、耗时和环境版本。
- **证据日志**：截图、请求响应、报告草稿、人工修改和最终结论。

回滚要覆盖代码与模型工程。Prompt、Skill、检索库、评估集、工具版本和权限策略都要版本化。若某次迭代让误报上升或触发越权风险，团队应能回到上一版配置。

6 评估：从演示效果到生产可信

6.1 评估对象

评估不能只看模型回答是否漂亮。主题 08 的评估对象包括模型、工具链、上下文管理和交付结果。

评估对象	核心问题	证据
------	------	----

模型 <i>M</i>	是否理解安全任务、代码和任务语境	标注样例、人工评分、失败案例
数据 <i>D</i>	是否覆盖真实漏洞、误报和任务场景	样本来源、版本、质量检查记录
工具链	是否正确调用工具并解释结果	调用日志、参数、输出、复现记录
上下文管理	是否保留关键细节并去除噪声	压缩前后对比、漏失字段清单
交付物	是否能被客户和工程团队执行	报告审核记录、修复反馈、复测结果

6.2 指标组合

安全工程的指标要同时覆盖质量、效率和风险：

$$Deliverability = \frac{Verified\ Findings}{Total\ Findings} \times Evidence\ Completeness \times Review\ Pass\ Rate$$

公式中的四项分别表示已验证发现数、总候选发现数、证据完整度和人工审核通过率。该指标并非唯一标准，却能提醒团队：交付能力来自真实发现、完整证据和人工审核共同作用。

实务中还应跟踪：

- **误报率**：报告候选中被人工驳回的比例；
- **漏报回溯**：事后发现系统没抓住的关键问题；
- **工具失败率**：工具调用失败、参数错误或权限阻断的比例；
- **人工接管率**：需要人类介入的步骤比例；
- **交付周期**：从任务开始到报告完成的时间。

7 开放工作台与生态

7.1 开放的两层含义

演讲者把工作台描述成“开放的白帽工作台”。开放有两层含义。第一层是能力开放：平台可以接入外部 MCP、Skill、Agent 和传统工具。第二层是生态开放：白帽可以把日常经验沉淀成 Skill，上传、共享、复用，形成共同进化的知识闭环。

这对工程系统提出更高要求。开放能力会带来工具来源复杂、接口标准不一、权限边界模糊、质量参差和供应链风险。开放生态会带来版本治理、评价机制、审核机制和撤回机制。开放像城市道路网，连接越多，交通越便利；道路标识、红绿灯、限速和事故处理机制也必须一起增长。

7.2 工作台的交付形态

一个面向白帽的 AI workbench 至少要覆盖五类交付对象：

1. **任务计划**：目标、范围、工具、步骤、风险点和人工接管点。
2. **执行过程**：浏览器操作、抓包、扫描、代码分析和工具调用。
3. **证据包**：截图、请求响应、日志、代码位置、复现材料。
4. **判断结果**：漏洞候选、影响分析、误报排除、优先级。
5. **报告与沉淀**：客户报告、复测记录、Skill 更新和评估样例。

这五类对象构成交付链。只有回答“输入是什么、处理过程是什么、输出是什么、证据在哪里、谁复核过”，系统才能从演示走向交付。

8 交付清单

8.1 输入清单

- 授权范围：目标域名、代码仓库、接口、测试窗口和禁止动作。
- 业务上下文：功能说明、角色权限、关键流程、历史问题。
- 数据材料：抓包、日志、代码、截图、历史报告、评估样例。
- 工具清单：浏览器、代理、扫描器、MCP、Skill、报告模板。
- 风险策略：权限层级、审批规则、敏感字段、回滚方式。

8.2 处理清单

- 任务拆解是否写入可审计计划；
- 工具调用是否记录参数、输出和错误；
- 上下文压缩是否保留安全关键字段；
- 高风险动作是否触发审批；
- 候选结论是否有证据绑定；
- 失败路径是否写入复盘材料。

8.3 输出清单

- 漏洞候选列表与验证状态；
- 每个发现的证据、影响、复现条件和修复建议；
- 误报排除理由和剩余不确定性；
- 工具调用 trace 与审批记录；
- 报告版本、复核人和发布时间；
- 可沉淀为 Skill 或评估样例的经验。

交付判断

若系统只能给结论，交付风险很高；若系统能给结论、证据、路径、审批和回滚点，它才具备进入安全工程流程的资格。

9 官方参考

本节只使用 `official_references.md` 收录且主题 08 推荐的官方资料。它们用于校准工程术语，正文主线仍以本主题演讲材料为核心。

- Anthropic, *Building effective agents*. <https://www.anthropic.com/engineering/building-effective-agents>
该文区分 workflow 与 agent, 并总结 prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer 等模式；对本章的启发是：工程系统需要简单、透明、工具接口文档化和测试。
- OpenAI, *New tools for building agents*. <https://openai.com/index/new-tools-for-building-agents>
该文把 agent 描述为代用户完成任务的系统，并强调工具、编排、观测、文件/网络/计算机使用能力和 tracing；对本章的启发是：可交付系统要能观察多步工作流，而非只保存最终回答。

10 配图接口

11 结语

主题 08 的结论可以压缩成一个工程公式：

$$System = M + D + Tools + Workflow + Evaluation + Controls + Human Review$$

其中 M 是模型， D 是数据，Tools 是工具，Workflow 是工作流，Evaluation 是评估，Controls 是权限、日志、回滚和审计控制，Human Review 是白帽判断。模型让系统拥有理



图 1: 主题 08 发布与定位页: 左图补抓发布会早段的任务执行演示入口, 右图承接 DigPool AI / AI Agent Workbench Preview 的产品定位。两张图共同说明模型工程的起点是可交付的工作台: 模型 M 、任务目标、工具入口和执行证据需要放在同一个操作面板里。²

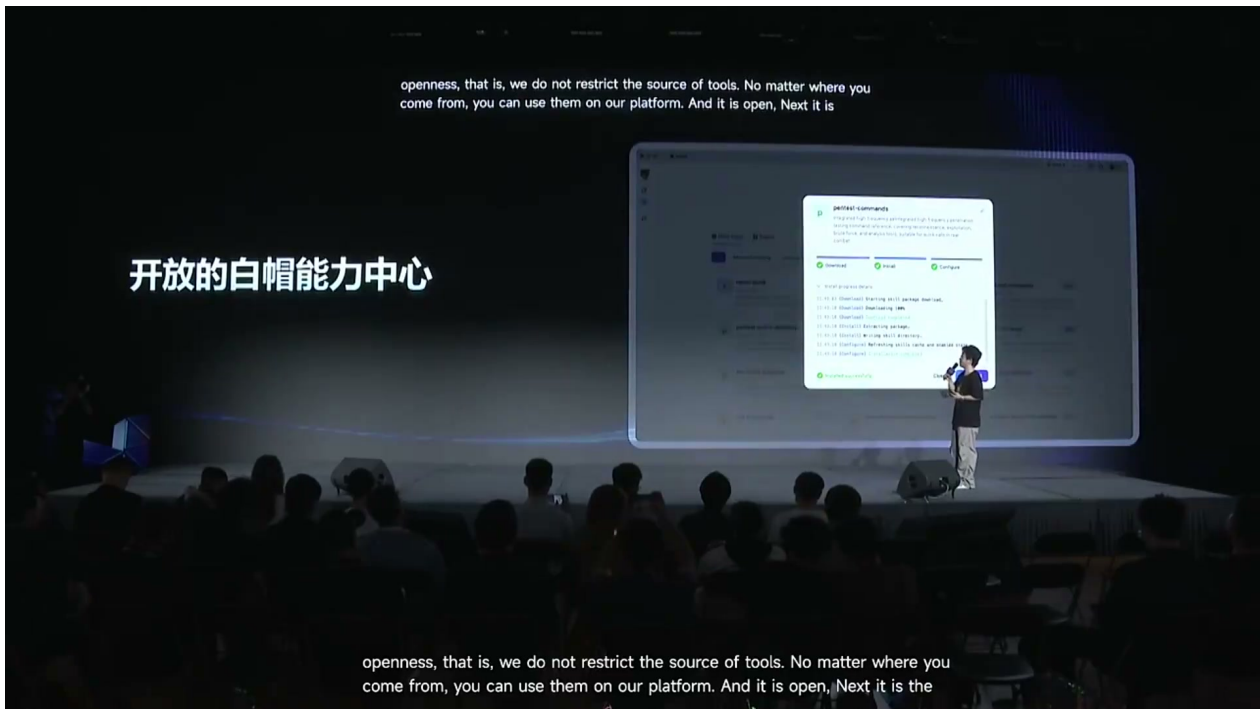


图 2: 开放的白帽能力中心: 画面强调工具来源开放、能力可接入。这里的关键语义是“能力中心”像安全工程里的插槽总线, MCP、Skill、Agent 和传统测试工具可以接入, 但每次接入都要被权限、日志与执行边界约束。⁴

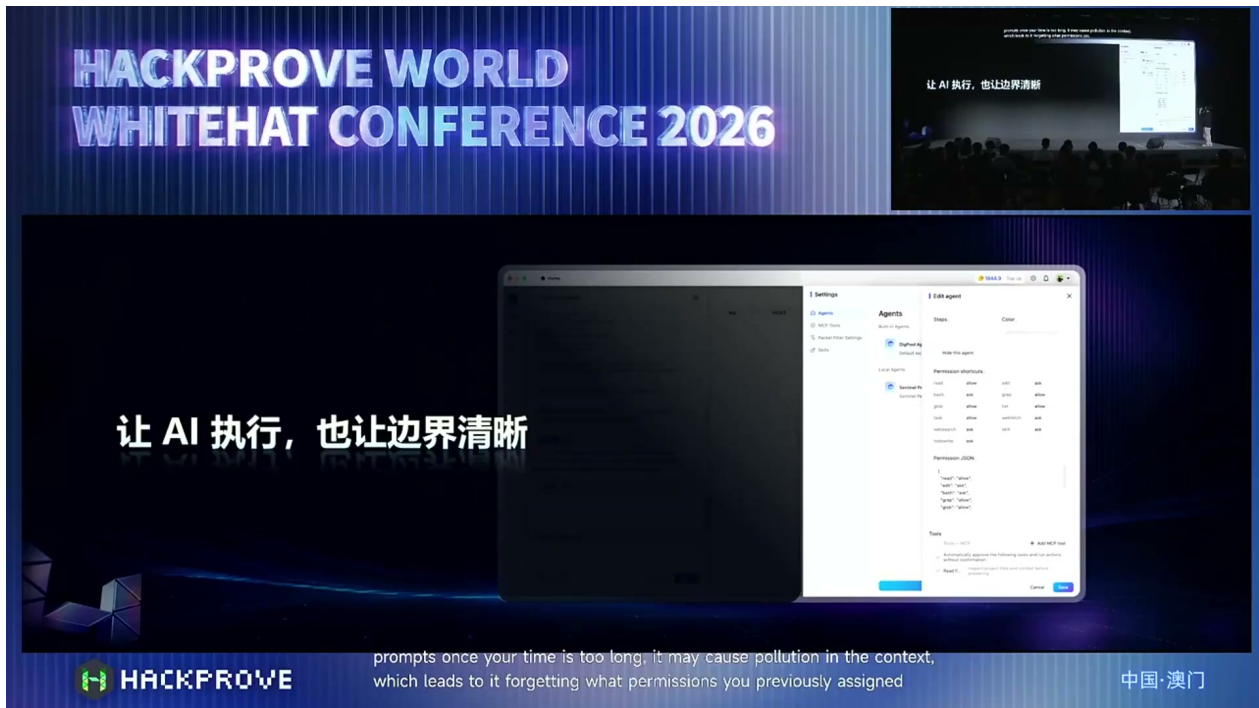


图 3: 让 AI 执行, 也让边界清晰: 该页把自动执行与权限边界放在同一屏, 适合解释工程系统里的控制面。若把 Agent 看成会调用函数的执行器, 安全工程就要明确哪些工具可调、何时需要人工确认、哪些输出必须落到证据 E 中。⁶

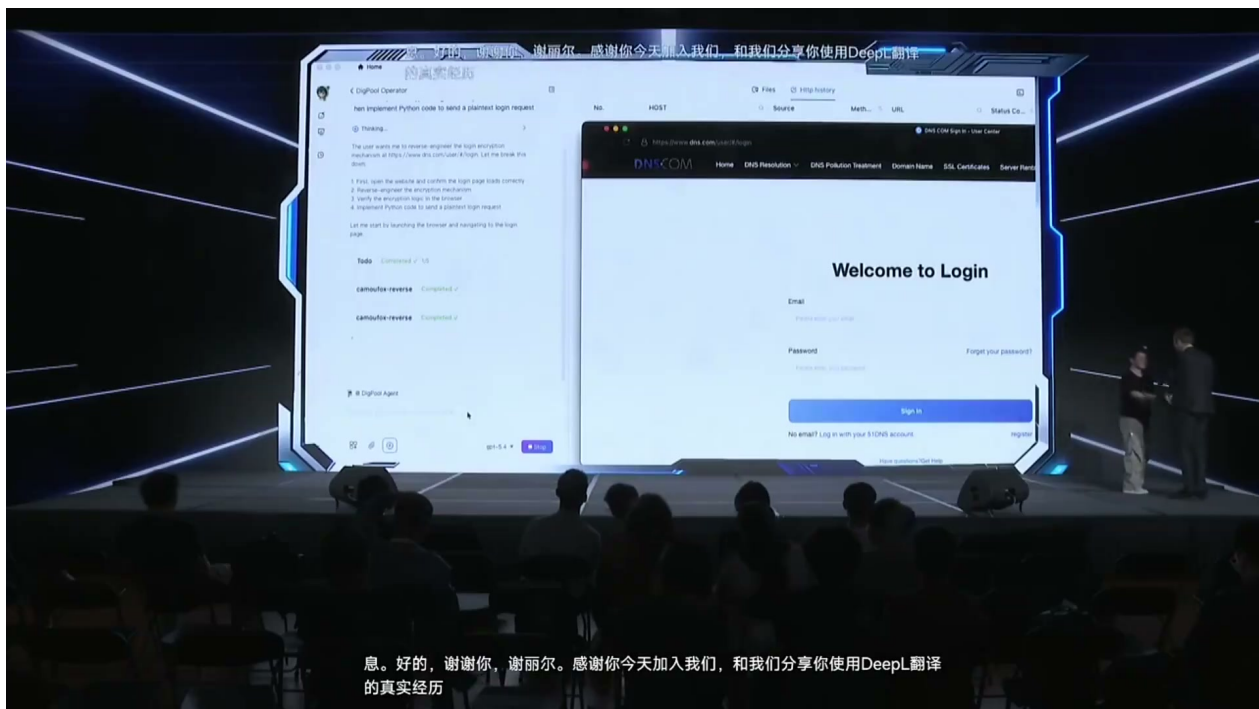


图 4: 登录与目标系统演示页: 画面进入真实 Web 目标的登录界面, 说明模型工程已经从“会回答”推进到“能在受控环境中完成测试流程”。输入、处理、输出、留证四个环节在这里变成可观察的操作链。⁸

解和生成能力；数据让系统贴近真实场景；工具让系统能行动；评估让系统知道自己是否做对；控制让系统在边界内运行；人工复核让责任留在可问责的人类流程中。

从会用模型到能交付系统，真正跨过去的是工程门槛。安全团队需要少一点热闹演示，多一点能被授权、能被记录、能被复核、能被回滚、能被客户使用的系统。这个门槛越早补齐，AI 在白帽工作中的价值越能持续释放。